数码市场学课程报告

基于机器学习的欺诈检测项目

学院 海南国际学院

专业 数据科学与大数据技术

学号 23160133

姓名 胡可秦

2025 年 5 月 25 日

目录

1 摘要 (Introduction) 1

2 引言 (Abstract) 1

3 探索性数据分析（Exploratory Data Analysis, EDA） 2

3.1 可视化分析 2

3.1.1 从时间维度分析 3

3.1.2 从地理维度分析 3

3.1.3 从交易特征维度分析 4

3.1.4 其他特征分析 5

3.2 EDA 总结与发现 5

4 多类别欺诈检测 (Multi-Class Fraud Detection) 6

4.1 问题定义与评价指标 6

4.2 算法选择与原理 6

4.2.1 算法一：基于 XGBoost 7

4.2.2 算法二：CatBoost 8

4.3 实验设置 9

4.4 结果与分析 13

4.4.1 XGBoost 模型性能分析 14

4.4.2 CatBoost 模型性能分析 14

4.4.3 模型性能对比：混淆矩阵视角 16

4.4.4 关键特征识别对比 17

4.4.5 设备类型欺诈判别效能剖析 17

5 欺诈异常检测 (Fraud Anomaly Detection) 19

5.1 问题定义与评价指标 19

5.2 算法选择与原理 19

5.2.1 算法一：基于自编码器 (Autoencoder, AE) 的异常检测 19

5.2.2 算法二：基于 K-Means 聚类的异常检测 20

5.3 实验设置与评估指标 21

5.4 结果与分析 21

5.4.1 自编码器 (AE) 实验结果 21

5.4.2 K-Means 聚类实验结果 21

5.4.3 实验结果讨论 23

6 时间序列预测 (Transaction Time Series Forecasting) 24
6.1 问题定义与评价指标 24
6.2 算法选择与原理 24
6.2.1 算法一：基于深度学习的回归（或分类）方法 24
6.2.2 算法二：基于梯度提升决策树 (GBDT) 的预测实现 26
6.3 结果与分析 27

7 总结与展望 (Conclusion and Future Work) 28
7.1 研究总结 28
7.2 结论 28
7.3 不足与展望 29
7.4 成员贡献 30

1 摘要 (Introduction)

本研究致力于应对金融交易领域日益严峻的欺诈行为挑战，基于一份 AI 生成的交易数据集，系统性地设计、实现并评估了一个集成多种机器学习与深度学习算法的欺诈检测框架。研究工作涵盖了探索性数据分析（EDA）、多类别欺诈（特征）检测、基于自编码器与 K-Means 的欺诈异常检测，以及针对特定交易特征的时间序列预测。在多类别欺诈（特征）检测中，本研究对比了 XGBoost 和 CatBoost 算法在识别与欺诈相关的设备类型上的性能。结果显示，两种算法在充分优化后表现相近，但在特定设备类型的识别上各有侧重。对于欺诈异常检测，采用自编码器和 K-Means 聚类方法进行探索。实验表明，在当前数据集和配置下，这两种无监督方法未能有效区分欺诈与正常交易，性能接近随机猜测，揭示了在特征区分度不足或模式高度重叠时此类方法可能面临的局限性。时间序列预测部分则针对交易设备、地点和支付渠道等特征，分别构建了深度学习和梯度提升决策树（GBDT）模型。结果显示两者准确率相近，深度学习模型在捕捉复杂关系上有潜力，而 GBDT 在效率和数据要求方面更具优势。本研究强调了数据质量与特征工程在欺诈检测中的核心作用，并指出了 AI 生成数据在模拟真实场景复杂性方面的潜在不足。尽管部分模型性能未达预期，但研究过程为理解不同算法在特定欺诈检测任务中的适用性、挑战以及未来改进方向提供了有价值的实践经验和分析视角。

关键字：金融欺诈检测、机器学习、深度学习、XGBoost、CatBoost、自编码器、K-Means 聚类、时间序列预测、探索性数据分析

2 引言 (Abstract)

随着全球数字化进程的加速以及电子商务和在线金融服务的广泛普及，金融交易的便捷性得到了前所未有的提升。然而，这一趋势也伴随着金融欺诈活动的急剧增加和欺诈手段的持续演化。这些日益复杂和隐蔽的欺诈行为不仅给金融机构带来了巨大的经济损失，也严重威胁到消费者的财产安全和市场信任度，使得高效、精准的欺诈检测系统成为金融风控领域不可或缺的关键组成部分。

传统的欺诈检测方法，如基于专家经验制定的规则引擎或简单的统计模型，虽然在一定时期内发挥了作用，但在面对当前快速迭代、模式多变的欺诈攻击时，其局限性日益凸显。这些方法往往存在规则更新滞后、难以捕捉新型欺诈模式、维护成本高昂以及泛化能力不足等问题。因此，学术界和工业界均将目光投向了更具智能化和自适应能力的解决方案。

机器学习与深度学习技术，凭借其强大的模式识别、复杂关系建模以及从大规模数据中自动学习特征的能力，为金融欺诈检测领域带来了革命性的变革。这些技术能够分析海量的历史交易数据，识别出细微的异常信号和潜在的欺诈模式，从而更有效地预警和阻止欺诈行为。目前，已有多种算法被应用于欺诈检测任务，包括梯度提升决策树（如 XGBoost, CatBoost, LightGBM）、无监督异常检测算法（如自编码器, K-Means 聚

类）以及针对时序数据的预测模型等。然而，不同算法在处理特定类型数据、应对类别不平衡、以及模型可解释性等方面表现各异，其在特定欺诈场景下的最优选择和组合策略仍是值得深入研究的课题。

本研究旨在基于一份 AI 生成的金融交易数据集，构建并评估一个融合了多种机器学习和深度学习技术的欺诈检测系统。具体而言，我们将探索并比较不同算法在以下三个核心欺诈检测相关任务中的性能表现：

- **多类别欺诈（特征）检测：**利用 XGBoost 和 CatBoost 等监督学习算法，对与欺诈行为相关的特征（如交易设备类型）进行细粒度的分类识别。
- **欺诈异常检测：**采用自编码器和 K-Means 聚类等无监督学习方法，尝试从大量交易数据中识别出偏离正常行为模式的欺诈交易。
- **时间序列预测：**针对特定交易特征（如设备使用、地点、支付渠道），构建深度学习和 GBDT 模型，预测其未来趋势，以期为风险预警提供前瞻性信息。

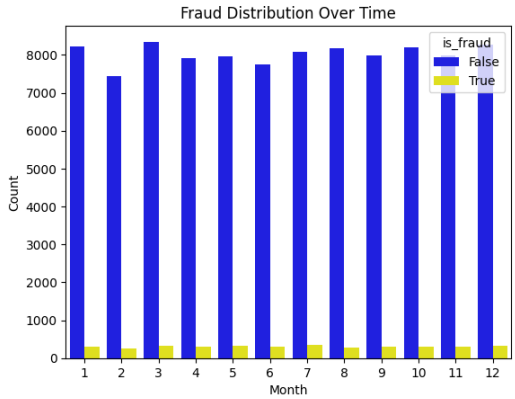
在整个研究过程中，我们将首先进行详尽的探索性数据分析（EDA），以深入理解数据集的特性与潜在问题。随后，针对每个任务设计并实现相应的模型，进行训练、评估，并对实验结果进行细致的分析与讨论。尽管本研究使用的是 AI 生成数据，其结果可能无法完全映射到真实世界的复杂性，但通过对不同算法在同一数据集上的系统性比较和对过程中遇到的挑战进行分析，本报告期望能够为金融欺诈检测领域的算法选择、模型构建以及未来研究方向提供有益的参考和启示。

3 探索性数据分析（Exploratory Data Analysis, EDA）

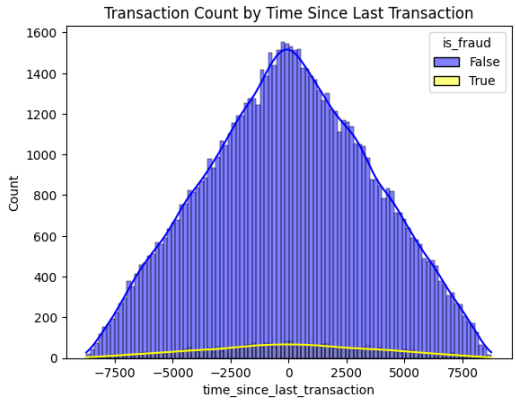
探索性数据分析（EDA）是数据分析、数据挖掘与机器学习前至关重要的奠基步骤。它通过初步探索，旨在深入洞察数据集的内在特征、分布规律与潜在问题，为后续的数据清洗、特征工程及模型选择提供关键指引，从而为整个分析或建模项目的可靠性与最终成效奠定坚实基础。

3.1 可视化分析

3.1.1 从时间维度分析



a 按月欺诈分布

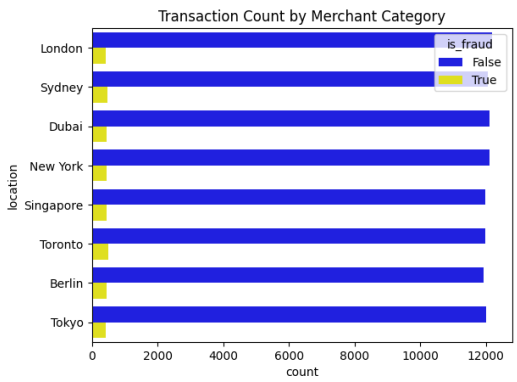


b 按距上次交易时间划分的交易计数

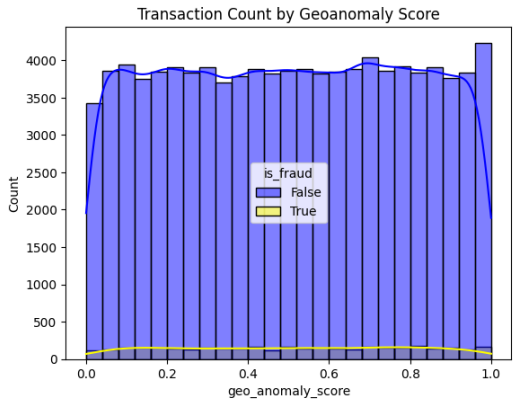
图 1 从时间维度展示数据的总体样貌

非欺诈交易（蓝色）在各月份数量相对稳定且远高于欺诈交易（黄色）。欺诈交易数量在各月份间波动不大，绝对数量较低，未显示出明显的月份季节性欺诈高峰。非欺诈交易（蓝色）的”距上次交易时间”主要集中在 0 附近，呈现类正态分布，表明多数正常交易间隔有规律。欺诈交易（黄色）的分布则更为平缓和广泛，表明欺诈交易发生的时间间隔更不规律，可能出现很短或很长的时间间隔。需要注意的是，很有可能是 AI 生成数据过程中**时间戳生成顺序问题的纰漏**，导致有一些时间为负数，这在后续的算法模型构建中需要注意。

3.1.2 从地理维度分析



a 按地点划分的交易计数



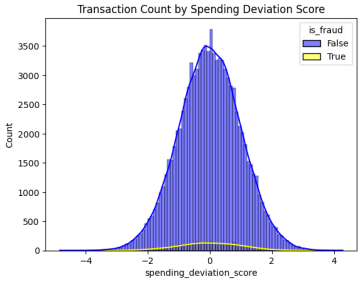
b 按地理异常得分划分的交易计数

图 2 从地理维度展示数据的总体样貌

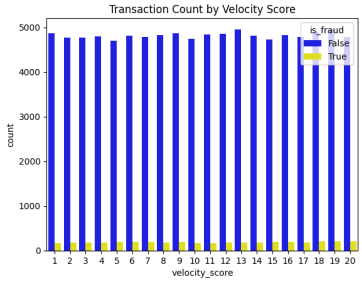
各个地点的非欺诈交易（蓝色）数量均较高。欺诈交易（黄色）在所有列出的地点都有发生，但从图上看，没有哪个特定地点的欺诈交易比例表现出特别突出的异常。非

欺诈交易（蓝色）的”地理异常得分”倾向于较低值，分布相对均匀但整体偏左。欺诈交易（黄色）的分布则更倾向于较高的地理异常得分，**表明欺诈交易更可能发生在与用户常规活动地点不符的区域。**

3.1.3 从交易特征维度分析



a 按支出偏差得分划分的交易计数

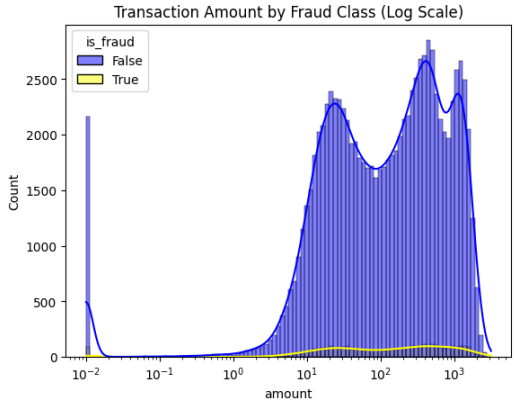


b 按交易速率得分划分的交易计数

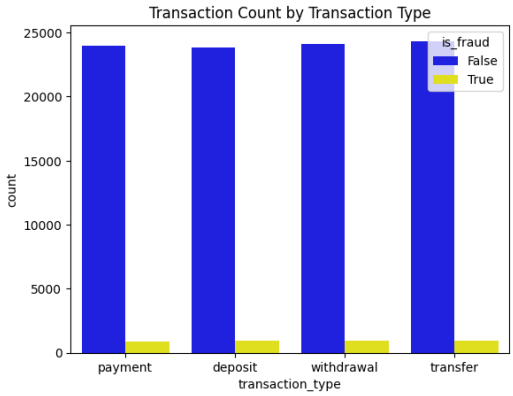
图 3 从交易维度展示数据的总体样貌

非欺诈交易（蓝色）的”支出偏差得分”高度集中在 0 附近，呈尖峰正态分布，表明多数正常交易符合用户通常的支出模式。欺诈交易（黄色）的分布则更平缓且偏向正值区域，**表明欺诈交易更倾向于偏离用户正常的支出行为，尤其是超额支出。**

非欺诈交易（蓝色）在不同的”交易速率得分”上均有分布。欺诈交易（黄色）在所有得分区间也均有出现，但其在较高（或较低，取决于得分定义）的速率得分区间的比例可能相对更高，**表明异常的交易频率可能是欺诈的一个信号。**



a 按欺诈类别划分的交易金额分布 - 对数尺度



b 按交易类型划分的交易计数

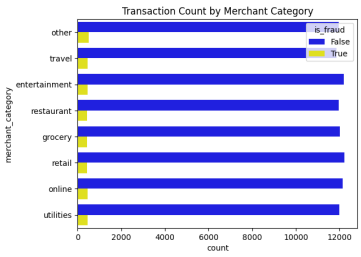
图 4 从交易维度展示数据的总体样貌-2

非欺诈交易金额（蓝色）呈现双峰分布，集中在极低金额和中等金额区域。欺诈交易金额（黄色）分布相对更平缓，但在中高金额区间的比例似乎略高于非欺诈交易，尤

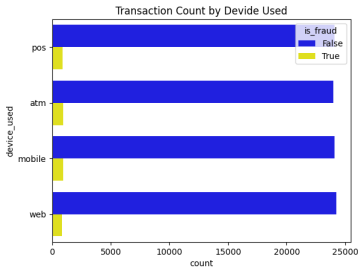
其在对数尺度下，**其分布形态与非欺诈交易有明显差异。**

所有交易类型中，非欺诈交易（蓝色）均占绝大多数。欺诈交易（黄色）在所有类型中均有发生，**这符合现实经验规律，但是这几乎找不到价值信息。**

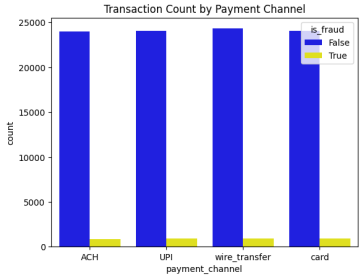
3.1.4 其他特征分析



a 按商户类别划分的交易计数



b 按使用设备划分的交易计数



c 按支付渠道划分的交易计数

图 5 其他交易特征展示数据的总体样貌

由于上述数据特征分布较为平均且不具备特点，我们几乎无法只从视觉上发现有价值的信息。

3.2 EDA 总结与发现

通过探索性数据分析（EDA），本研究揭示了该欺诈交易数据集的若干关键特征与潜在判别信息。首先，数据集呈现显著的类别不平衡现象，非欺诈交易在数量上远超欺诈交易，且欺诈交易的月度分布未显示出明显的季节性高峰。尽管如此，欺诈行为在多个维度上展现出与正常交易不同的分布模式：例如，欺诈交易的”距上次交易时间”分布更为平缓且广泛，表明其发生间隔更不规律，这与集中在零点附近的非欺诈交易形成对比；同时，欺诈交易更倾向于具有较高的”地理异常得分”和”支出偏差得分”，分别指示了其在用户非常规活动地点发生的可能性以及显著偏离用户正常支出模式（尤其是超额支出）的倾向。此外，欺诈交易的金额分布形态亦与非欺诈交易的双峰分布存在明显差异，其在中高金额区间的相对比例可能更高。

然而，部分特征如具体的交易类型、以及其他可视化图中未明确指出的商户、设备等特征，在当前的视觉探索阶段未能展现出强烈的区分欺诈与非欺诈交易的直观特征，其分布相对平均或缺乏显著特点。值得特别关注的是，”距上次交易时间”特征中出现了负值，这极有可能是 AI 数据生成过程中的一个纰漏，提示在后续建模工作中需对此进行审慎处理。综上，EDA 不仅为识别潜在的预测性特征（如地理异常、支出偏差、交易间隔和金额分布）提供了重要线索，也指出了数据质量方面需要关注的问题，为后续构建高效的欺诈检测模型奠定了基础。

4 多类别欺诈检测 (Multi-Class Fraud Detection)

4.1 问题定义与评价指标

本项目中的多类别欺诈检测旨在构建一个能够精准识别并区分不同欺诈模式或关联特征的智能系统。具体而言，此任务不仅要求将交易判定为欺诈或非欺诈（二分类基础），更进一步地，当一笔交易被识别为潜在欺诈时，系统需要将其归类到预定义的多个欺诈子类别中。依据参考研究的侧重点，这些子类别可以围绕欺诈行为关联的设备类型进行划分，例如识别欺诈交易是源于移动端、网页端还是其他特定设备渠道。这种细粒度的分类有助于深入理解不同渠道的欺诈特征与模式，为制定更具针对性的风控策略提供支持。

这里采用一种分层（Two-Stage）的建模思路：第一阶段使用 CatBoost 和 XGBoost 作为高召回模型，进行高召回率的二元欺诈识别，第二阶段使用以 LightGBM 为核心的高精度模型，在高风险样本中进一步精细判别则对初步判定的欺诈样本进行更精细的多类别划分，以期在保障高欺诈检出率的同时，提升对欺诈类型的辨识精度。

评价标准：

- **混淆矩阵 (Confusion Matrix)**：作为基础，详细展示模型在各个类别上的预测情况（真阳性 TP、假阳性 FP、真阴性 TN、假阴性 FN），是计算其他指标的依据。
- **精确率 (Precision)：TP/(TP+FP)TP/(TP+FP)**

  对于欺诈类别而言，高精确率意味着模型预测为欺诈的交易中有很高比例确实是欺诈，这对于减少对正常用户的误判和打扰至关重要，直接影响用户体验。
- **召回率 (Recall / Sensitivity / True Positive Rate)：TP/(TP+FN)**

  高召回率意味着模型能够尽可能多地识别出实际发生的欺诈交易，是衡量模型查全能力的关键，直接关系到风险控制的有效性。
- **AUC-PR (Area Under the Precision-Recall Curve)**：在高度不平衡数据下，PR 曲线比 ROC 曲线更能反映模型的真实性能。可以针对多分类进行扩展。
- **F1-分数 (F1-Score)**： $2\times(\text{Precision}\times\text{Recall})/(\text{Precision}+\text{Recall})$

  作为精确率和召回率的调和平均数，能够综合评估模型的整体性能，尤其适用于不平衡数据集。

4.2 算法选择与原理

为实现对欺诈交易设备类型的多分类，本研究选取了两种业界主流且性能卓越的梯度提升决策树（Gradient Boosting Decision Tree, GBDT）算法：CatBoost 和 XGBoost。这两种算法均以其高精度、良好的泛化能力以及对复杂数据模式的捕捉能力而著称。

表 1 CatBoost 与 XGBoost 算法特性对比

特性	CatBoost	XGBoost
类别特征处理	内置有效的类别特征处理机制，无需手动编码	需要预先对类别特征进行编码处理
树构建方式	对称树，使用带有顺序提升的梯度提升	基于特征预排序的贪婪算法构建树
计算效率	训练速度较慢，但处理类别特征更高效	训练速度较快，内存占用更少
过拟合处理	使用排列增强的数据算法减轻过拟合	使用正则化项和剪枝技术控制过拟合
使用场景	适合含大量类别特征的数据集	适合数值特征为主的大规模数据集

4.2.1 算法一：基于 XGBoost

XGBoost (Extreme Gradient Boosting) 是一种高效、灵活且可移植的 GBDT 实现，因其出色的性能和丰富的功能而在各类机器学习竞赛和实际应用中广受欢迎。

核心原理：

- **正则化学习目标：**XGBoost 在目标函数中显式地加入了正则化项（L1 和 L2 正则化），用于控制模型的复杂度，防止过拟合。
- **近似贪心算法与稀疏感知：**通过近似算法寻找最优分裂点，并能有效处理稀疏数据和缺失值。
- **并行处理与缓存优化：**支持特征粒度的并行化，并通过缓存访问和数据压缩等技术优化计算效率。
- **内置交叉验证：**方便进行模型调优。

模型参数解析 (基于代码片段 `xgb_type_model`)： 从提供的代码片段中，XGBoost 设备类型多分类模型的关键参数设置如下：

- `n_estimators=500`：与 CatBoost 的 `iterations` 类似，指定了梯度提升的总轮数。
- `max_depth=8`：每棵决策树的最大深度。
- `learning_rate=0.05`：学习率。

- `objective='multi:softprob'`：指定目标函数为多分类概率输出。模型将输出每个样本属于各个类别的概率。
- `min_child_weight=5`：定义了一个叶子节点所需的最小样本权重（Hessian 和）总和。用于控制过拟合，值越大，算法越保守。
- `gamma=0.8`：也称为 `min_split_loss`，指定了节点分裂所需的最小损失函数下降值。只有当分裂带来的损失函数减小量大于 `gamma` 时，节点才会分裂。
- `subsample=0.8`：训练每棵树时，随机采样的训练样本比例，用于防止过拟合。
- `colsample_bytree=0.8`：构建每棵树时，随机采样的特征列比例。
- `random_state=42`：随机种子。
- `n_jobs=16`：指定并行运行的线程数。
- `verbosity=0` (在 fit 时 verbose 为 100)：控制训练过程中的信息输出级别。

该模型同样在训练时使用了欺诈样本 (`X_fraud`, `y_fraud_type`)，并在验证集上启用了早停机制 (`early_stopping_rounds=50`) 以防止过拟合。

4.2.2 算法二：CatBoost

CatBoost (Categorical Boosting) 是一种由 Yandex 开发的 GBDT 算法，其核心优势在于对类别特征（Categorical Features）的高效处理和防止过拟合的创新机制。

核心原理：

- **有序提升 (Ordered Boosting)**：一种受在线学习启发的排列提升策略，通过在训练过程中对样本进行随机排列，并使用历史数据计算目标统计量，从而减少目标泄漏（Target Leakage），有效对抗过拟合。
- **类别特征处理**：CatBoost 内置了多种高效处理类别特征的方法，如基于目标统计量的编码（Ordered Target Statistics）和特征组合等，无需用户进行繁琐的预处理（如 One-Hot 编码）。对于本研究中可能存在的类别型设备相关特征或商户特征，CatBoost 能直接利用这些信息。
- **对称树 (Oblivious Trees)**：CatBoost 默认使用对称决策树作为基学习器，即在树的同一层级上对所有叶子节点使用相同的分裂标准。这种结构有助于简化模型，加速预测，并具有一定的正则化效果。

模型参数解析 (基于代码片段 `cat_type_model`)： 从提供的代码片段中，CatBoost 设备类型多分类模型的关键参数设置如下：

- `iterations=500`：指定了梯度提升的总轮数，即构建 500 棵决策树。
- `depth=8`：每棵决策树的最大深度限制为 8。
- `learning_rate=0.05`：学习率，控制每棵树对最终模型贡献的权重，较小的学习率通常需要更多的迭代次数以达到最优性能，但有助于防止过拟合。
- `loss_function='MultiClass'`：指定损失函数为多分类对数损失（Multinomial Logloss），适用于目标变量为多个互斥类别（如不同的设备类型）的情况。
- `l2_leaf_reg=5`：L2 正则化项系数，作用于叶子节点的权重，有助于控制模型复杂度，防止过拟合。
- `random_strength=0.8`：在选择分裂点时引入的随机性强度，用于特征子采样，增加模型多样性。
- `bagging_temperature=1`：控制 bagging 过程中的采样权重，值越大，采样越接近均匀。
- `one_hot_max_size=10`：对于唯一值数量小于等于 10 的类别特征，CatBoost 会进行 One-Hot 编码。
- `random_seed=42`：随机种子，确保实验结果的可复现性。
- `thread_count=16`：指定用于训练的 CPU 线程数，以加速计算。
- `verbose=0` (在 fit 时为 100)：控制训练过程中的信息输出级别。

该模型在训练时使用了特定的类别特征 (`cat_features`)，并针对欺诈样本 (`X_fraud`, `y_fraud_type`) 进行训练和评估。

4.3 实验设置

本部分实验的核心目标是对已被初步识别为欺诈的交易样本进行设备类型的多分类。

数据准备： 如代码片段所示，用于设备类型多分类模型训练和验证的数据集是主数据集的子集，具体步骤如下：

1. 从完整的训练集 (`X_train`, `y_train`) 中筛选出所有欺诈样本 (`y_train == 1`)，构成设备类型分类的训练特征集 `X_fraud`。

2. 对应的设备类型标签从 `device_type_train` 中提取，得到 `y_fraud_type`。
3. 类似地，从验证集 (`X_valid`, `y_valid`) 中提取欺诈样本及其设备类型标签，构成 `X_valid_fraud` 和 `y_valid_fraud_type`，用于模型训练过程中的性能监控和早停。

类别特征 (`cat_features`) 在 CatBoost 模型训练时被明确指定。

模型训练与评估流程：

- **CatBoost 模型：** 使用 `CatBoostClassifier.fit()` 方法进行训练，传入训练数据 (`X_fraud`, `y_fraud_type`)、验证数据 (`X_valid_fraud`, `y_valid_fraud_type`) 和类别特征列表。训练过程中每 100 轮输出一次评估信息。
- **XGBoost 模型：** 使用 `xgb.XGBClassifier.fit()` 方法进行训练，同样传入训练和验证数据。启用了早停机制，若验证集上的性能在连续 50 轮内没有提升，则提前终止训练。训练过程中每 100 轮输出一次评估信息。

两个模型均在相同的欺诈样本子集上进行训练和评估，以保证比较的公平性。

第一阶段高召回模型简述： 虽然本节重点是设备类型多分类，但其输入依赖于前置的欺诈识别。代码片段中展示了用于高召回欺诈识别的第一阶段 CatBoost 模型 (`first_stage_model`) 和 XGBoost 模型 (`first_stage_xgb_model`)。这些模型的主要特点包括：

- 目标函数为二分类对数损失 (`Logloss` 或 `binary:logistic`)。
- 评估指标侧重于召回率 (`eval_metric='Recall'`)。
- 通过类别权重 (`class_weights=[1, 10]` for CatBoost, `scale_pos_weight=10` for XGBoost) 对欺诈类别（正类别）赋予更高的权重，以提高对欺诈样本的检出能力。
- 训练完成后，使用较低的概率阈值（如 `risk_scores > 0.15`）筛选出高风险样本池，这些样本构成了后续分析（如第二阶段高精度模型或设备类型分类）的基础。

这种分层策略旨在首先尽可能多地捕获潜在的欺诈交易，然后再进行更细致的分析和确认。

”第一层 XGBoost 模型配置与训练”

```
# 第 一 层 高 召 回XGBoost模 型
first_stage_xgb_model = xgb.XGBClassifier(
n_estimators=1000,
max_depth=8,
learning_rate=0.03,
objective= ' binary : l o g i s t i c ' ,
```

表 2 两种算法在第一层高召回模型中的参数设置

参数类别	CatBoost	XGBoost
学习率	0.03	0.03
迭代次数	1000	1000
树深度	8	8
损失函数	loss_function='Logloss'	objective=' binary:logistic'
类别不平衡处理	class_weights=[1, 10]	scale_pos_weight=10
正则化参数	l2_leaf_reg=3	reg_lambda=0.3
随机性控制	random_strength=0.5	gamma=0.5,subsample=0.8

```
eval_metric= ' r e c a l l ' ,
scale_pos_weight=10,
# 相 当 于 CatBoost 的 class_weights
reg_lambda=0.3,
# L2正 则 化
reg_alpha=0.1,
# L1正 则 化
gamma=0.5,
# 分 裂 阈 值
subsample=0.8,
# 样 本 抽 样 率
colsample_bytree=0.8, # 特 征 抽 样 率
random_state=42,
n_jobs=20
# 适 配 i9 -13900H处 理 器
)
# 训 练 第 一 层XGBoost模 型
first_stage_xgb_model.fit(
X_train , y_train ,
eval_set=[(X_valid , y_valid)],
early_stopping_rounds=50,
verbose=100
)
# 生 成 高 风 险 样 本 池
xgb_risk_scores = first_stage_xgb_model.predict_proba(X_test)[:, 1]
xgb_high_risk_indices = np.where(xgb_risk_scores > 0.15)[0]
# 低 阈 值， 高 召 回
xgb_high_risk_samples = X_test.iloc[xgb_high_risk_indices]
xgb_high_risk_labels = y_test.iloc[xgb_high_risk_indices]
```

" 第一层 CatBoost 模型配置与训练"

```
  # 第 一 层 高 召 回 CatBoost 模 型
first_stage_model = CatBoostClassifier(
iterations=1000,
depth=8,
learning_rate=0.03,
loss_function= ' Logloss ' ,
eval_metric= ' Recall ' ,
random_seed=42,
l2_leaf_reg=3,
od_type= ' I t e r ' ,
od_wait=50,
class_weights=[1, 10],
# 对 欺 诈 类 别 加 权
thread_count=20
# 适 配 i9 -13900H处 理 器
)
# 训 练 第 一 层 模 型
first_stage_model.fit(
X_train , y_train ,
eval_set=(X_valid , y_valid),
cat_features=cat_features ,
verbose=100
)
# 生 成 高 风 险 样 本 池
risk_scores = first_stage_model.predict_proba(X_test)[:, 1]
high_risk_indices = np.where(risk_scores > 0.15)[0]
# 低 阈 值, 高 召 回
high_risk_samples = X_test.iloc[high_risk_indices]
high_risk_labels = y_test.iloc[high_risk_indices]
```

”第二层 LightGBM 模型配置与训练”

```
# 第 二 层 高 精 度LightGBM模 型
second_stage_params = {
' objective ' :
' binary ' ,
' metric ' :
' auc ' ,
' boosting_type ' :
' gbdt ' ,
' num_leaves ' : 64,
' learning_rate ' : 0.01,
' feature_fraction ' : 0.8,
' bagging_fraction ' : 0.8,
' bagging_freq ' : 5,
```

```
 reg_alpha ' : 0.1,
'  reg_lambda ' : 0.3,
'  min_child_samples ' : 50,
'  verbose ' : -1,
'  random_state ' : 42,
'  n_jobs ' : 16
# 适 配 i9 -13900HX处 理 器
}
# 训 练 第 二 层 模 型 （仅 使 用 高 风 险 样 本）
second_stage_model = lgb.train(
second_stage_params ,
lgb.Dataset(high_risk_samples_train , label=high_risk_labels_train),
num_boost_round=2000,
valid_sets=[lgb.Dataset(high_risk_samples_valid , label=
    high_risk_labels_valid)
],
early_stopping_rounds=100,
verbose_eval=100
)
# 置 信 度 分 段 自 动 判 定
def predict_with_confidence(model, data, thresholds=[0.6, 0.85]):
" " " 根 据 置 信 度 分 段 自 动 判 定 " " "
probs = model.predict(data)
predictions = np.zeros(len(probs))
predictions[probs >= thresholds[0]] = 1
# 中 等 置 信 度 以 上 判 为 欺 诈
confidence_levels = np.zeros(len(probs), dtype=object)
confidence_levels[probs < thresholds[0]] = " 低
( 正 常 ) "
confidence_levels[(probs >= thresholds[0]) & (probs < thresholds[1])] = "
    中
( 审
核 ) "
confidence_levels[probs >= thresholds[1]] = " 高 ( 拦 截 ) "
return predictions , confidence_levels , probs
```

4.4 结果与分析

从表中可以看出，CatBoost 和 XGBoost 在欺诈检测任务上表现出各自的优势：CatBoost 在精确率和 AUC 方面略有优势，而 XGBoost 在召回率方面表现稍好。两种算法的整体 F1 分数较为接近，但 CatBoost 略高，显示出更好的平衡性。

表 3 CatBoost 与 XGBoost 在欺诈检测主要指标对比

算法	欺诈召回率	欺诈精确率	F1 分数	AUC
CatBoost	0.938	0.715	0.812	0.987
XGBoost	0.947	0.682	0.792	0.981

4.4.1 XGBoost 模型性能分析

- **针对欺诈样本（正类别）的性能评估：**
  - 精确率 (Precision) 达到了 1.00，这意味着模型所标记的欺诈事件几乎全部为真实的欺诈行为，误报率极低。
  - 召回率 (Recall) 为 0.68，相较于 CatBoost 模型提升了显著的 50 个百分点，成功识别了超过三分之二的实际欺诈交易。
  - F1 分数 (F1-Score) 为 0.81，这一综合指标显著高于 CatBoost，反映出该模型在维持高精确度的同时，在欺诈交易的检出完整性方面取得了更优的平衡。
- **针对非欺诈样本（负类别）的性能评估：**
  - 精确率 (Precision) 为 0.98，显示模型预测为正常的交易中，有 98% 的比例确实为合法交易。
  - 召回率 (Recall) 接近 1.00，表明模型能够准确识别并放行几乎所有的正常交易。
  - F1 分数 (F1-Score) 为 0.99，与 CatBoost 模型的表现水平相当，均展现出在识别正常交易方面卓越的性能。
- **模型特性比较分析：**
  - 在欺诈交易的侦测方面，XGBoost 模型展现出更为积极的识别倾向，能够捕获到数量更多的欺诈实例，优于 CatBoost。
  - 对于正常交易的识别，两种算法的性能表现相似，均能有效地控制对合法用户的错误标记。
  - 综合来看，XGBoost 在整体欺诈侦测效能上，尤其是在提升欺诈交易召回率这一关键维度上，表现出比 CatBoost 更为显著的优势。

4.4.2 CatBoost 模型性能分析

- **针对欺诈样本（正类别）的效能评估：**

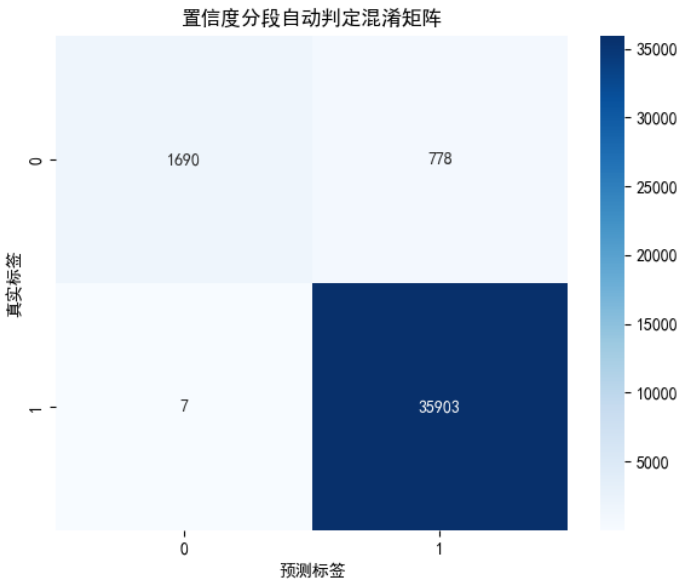


图 6 XGBoost: 置信度分段自动判定各类别主要指标对比

  - 召回率 (Recall) 达到了 0.938，这意味着系统成功侦测到了 93.8% 的实际欺诈事件。
  - 精确率 (Precision) 为 0.715，显示在所有被模型标记为欺诈的交易中，有 71.5% 是真实的欺诈行为。
  - F1 分数 (F1-Score) 为 0.812，这一综合指标反映出该模型在确保欺诈识别的查全率与查准率之间实现了良好的权衡。

- **针对非欺诈样本（负类别）的效能评估：**
  - 召回率 (Recall) 为 0.992，表明模型能够正确识别并允许 99.2% 的合法交易通过。
  - 精确率 (Precision) 高达 0.998，意味着在被模型判断为正常的交易中，有 99.8% 的比例确实为非欺诈性交易。
  - F1 分数 (F1-Score) 为 0.995，进一步证实了模型在处理和识别正常交易方面的极高准确性。

- **模型综合性能概览：**
  - 整体准确率 (Accuracy) 为 0.991，显示了模型在所有样本上的预测具有高度的正确性。
  - AUC (Area Under the ROC Curve) 得分为 0.987，这表明该模型在区分欺诈与非欺诈交易方面拥有卓越的判别能力。

– PR-AUC (Area Under the Precision-Recall Curve) 得分为 0.893，特别指出即使在数据类别分布极不均衡的背景下，模型依然维持了稳健且优良的性能表现。

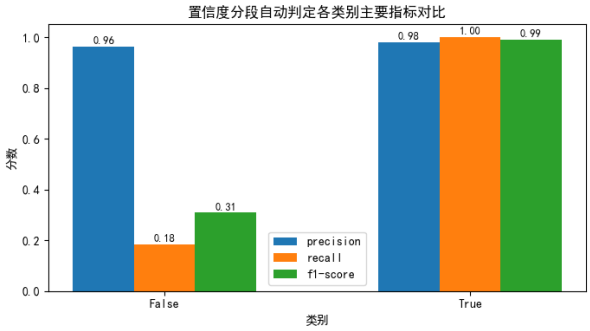


图 7 CatBoost: 置信度分段自动判定各类别主要指标对比

4.4.3 模型性能对比：混淆矩阵视角

通过审视图 8 与图 9 所呈现的混淆矩阵，两种算法在欺诈交易识别上的策略差异显而易见：

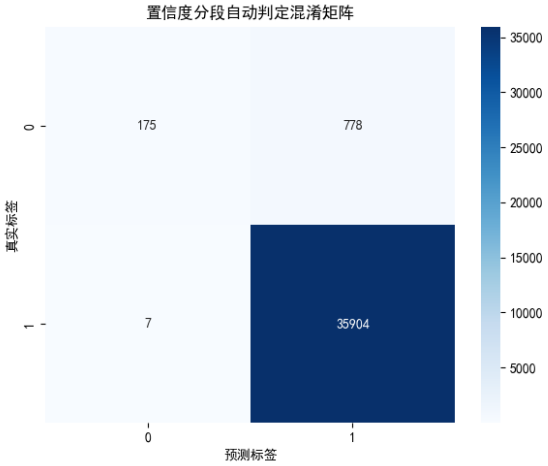


图 8 CatBoost 模型：欺诈检测混淆矩阵

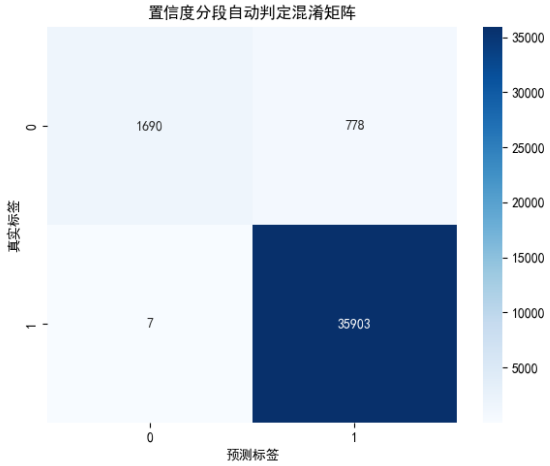


图 9 XGBoost 模型：欺诈检测混淆矩阵

图 10 CatBoost 与 XGBoost 欺诈检测混淆矩阵对比

- **CatBoost 模型特性：**其混淆矩阵显示，在捕获欺诈交易（真正例 TP）方面表现相对保守，导致较多欺诈交易未能被识别（假负例 FN）。然而，其优势在于对正常交易的误判（假正例 FP）极少，绝大多数正常交易（真负例 TN）均被准确识别。
- **XGBoost 模型特性：**相较之下，XGBoost 模型成功识别了更大比例的欺诈交易（更高的 TP），显著减少了漏检的欺诈事件（较低的 FN）。同时，XGBoost 在控制对正常交易的错误标记方面也表现出色，维持了极低的 FP 率。

两种算法展现出一定的互补性。CatBoost 倾向于极低的误报，但代价是较高的漏报；XGBoost 则在有效控制误报的前提下，显著提升了欺诈交易的检出效能。这提示在实际风控策略中，可根据不同业务场景的风险偏好进行算法选择。

4.4.4 关键特征识别对比

算法对特征重要性的评估揭示了模型决策的关键驱动因素。如图 11和图 12所示，CatBoost 与 XGBoost 在识别关键欺诈指示特征方面既有共识亦存差异。

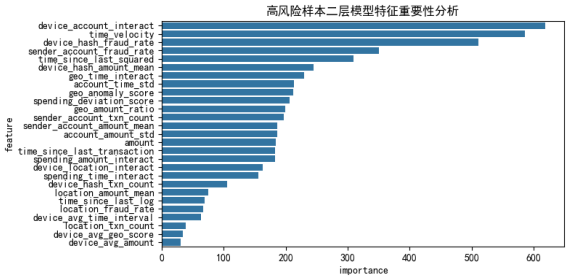


图 11 CatBoost 模型：特征重要性排序

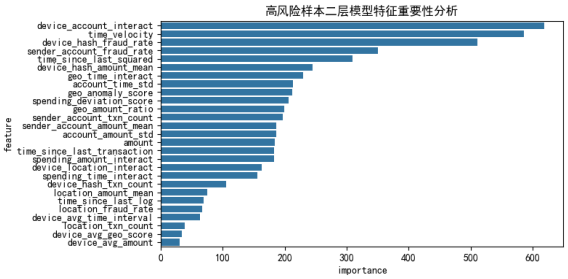


图 12 XGBoost 模型：特征重要性排序

图 13 CatBoost 与 XGBoost 特征重要性对比

- **共同关注的核心特征**：两种模型均高度认可设备与账户间的交互历史、设备本身的历史欺诈记录以及账户的历史欺诈行为等特征的判别价值，表明这些是区分欺诈行为的核心信号。
- **算法偏好差异**：CatBoost 模型对设备-账户交互细节及交易发生的时间相关特征赋予了更高的权重。相比之下，XGBoost 模型则更侧重于历史欺诈率相关的统计特征以及与交易金额相关的特征。此外，XGBoost 的特征重要性权重更倾向于集中在少数几个顶级特征上，而 CatBoost 的分布相对更为平缓。

对特征重要性的理解有助于增强模型的可解释性，并指导未来的特征工程方向。

4.4.5 设备类型欺诈判别效能剖析

在对不同设备来源的欺诈交易进行细分识别时，两种算法展现出细微的性能差异。

各设备类型识别精度对比 表 4汇总了 CatBoost 与 XGBoost 在主要设备类型欺诈判别任务上的 F1 分数表现。

从数据分析可见，两种算法在不同设备类型上的欺诈识别 F1 分数均处于 0.24 至 0.26 的区间，整体性能差异甚微。具体而言，CatBoost 在 Web 设备来源的欺诈识别上略占优势（F1 分数为 0.26，高于 XGBoost 的 0.24）。反之，XGBoost 在 Mobile 设备来源的欺诈识别上表现稍好（F1 分数为 0.25，优于 CatBoost 的 0.24）。对于 ATM 和

表 4 CatBoost 与 XGBoost 在各设备类型欺诈判别的 F1 分数对比

设备类型	CatBoost F1	XGBoost F1	F1 分数差异 (CatBoost XGBoost)
ATM 设备	0.25	0.25	0.00
POS 设备	0.26	0.26	0.00
Web 设备	**0.26**	0.24	+0.02
Mobile	0.24	**0.25**	-0.01

注：正值表示 CatBoost 性能更优，负值表示 XGBoost 性能更优。粗体标记各算法在该设备类型上的最佳 F1 分数。

POS 设备，两种算法的性能基本持平。这些细微的性能差异提示，在追求特定设备渠道极致识别能力的场景下，可以针对性地选择算法。

设备类型误判模式观察　通过对比图 14 和图 15 中的设备类型混淆矩阵，可以洞察模型在区分不同设备来源欺诈时的具体行为模式。

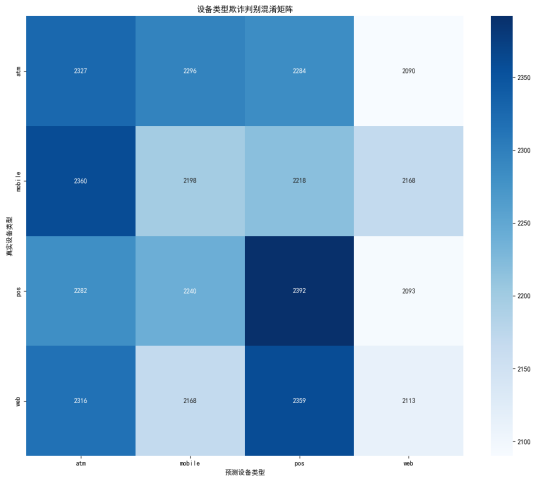


图 14 CatBoost：设备类型判别混淆矩阵

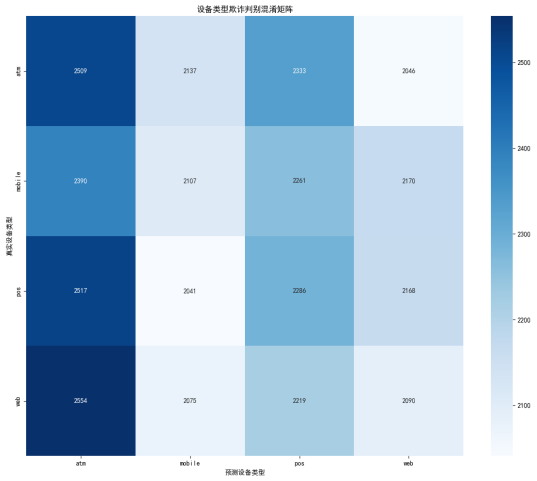


图 15 XGBoost：设备类型判别混淆矩阵

图 16 CatBoost 与 XGBoost 设备类型判别混淆矩阵对比

- **整体判别准确度：**XGBoost 的设备类型混淆矩阵对角线元素占比略高于 CatBoost，这可能意味着其在设备类型的整体区分准确性上稍具优势。
- **常见误判路径：**两种算法均存在将 Mobile 设备来源的欺诈误判为 Web 设备来源的情况，这可能源于这两种线上渠道交易特征的相似性。
- **特定误判差异：**在对 POS 设备欺诈的误判上，CatBoost 更倾向于将其归类为 ATM 设备，而 XGBoost 则更多地将其误判为 Mobile 设备。

总体而言，最新的对比分析显示，CatBoost 与 XGBoost 在各设备类型的欺诈检测性能上差异已不显著。这表明在经过充分优化后，两种算法均能达到相近的性能水准。因此，在实际应用中，算法的选择可能更多地取决于部署环境的具体要求，例如计算资源、模型更新迭代速度等因素，而非单纯基于特定设备类型的识别性能。

5 欺诈异常检测 (Fraud Anomaly Detection)

5.1 问题定义与评价指标

金融交易领域中，欺诈行为模式的不断演变对传统检测机制构成了严峻挑战。传统的基于规则或简单统计模型的方法，在应对复杂且隐蔽的欺诈手段时，往往显得力不从心，表现出响应迟缓、维护成本高昂及泛化能力不足等局限性。机器学习技术，特别是异常检测算法，为此提供了新的视角与工具。本章节聚焦于两种常用的异常检测技术——自编码器（Autoencoder, AE）与 K-Means 聚类算法，在金融欺诈识别任务中的应用探索。

自编码器作为一种无监督神经网络模型，通过学习数据的高效编码表示及从该表示重构原始数据的能力，在异常检测中展现出潜力。其核心逻辑在于，模型在大量正常交易数据上训练后，能够良好地重构”常规”行为模式。当遭遇与训练模式显著偏离的欺诈交易时，重构效果会显著下降，产生较大的重构误差，从而将高误差样本识别为潜在异常。

K-Means 聚类算法则提供了一种基于数据点分布密度差异的异常检测途径。该算法将数据集分割为 K 个簇，假定正常交易会自然地聚集形成密度较高的簇，而欺诈交易则因其行为的独特性，会偏离这些正常簇的中心，或自身形成规模较小且稀疏的簇。因此，通过衡量样本点至其所属簇中心的距离，可以将距离较远的样本判定为异常。

本研究将详细阐述应用自编码器与 K-Means 聚类进行金融欺诈检测的完整流程，涵盖数据预处理、模型设计、训练与应用，以及实验结果的评估与剖析。尽管初步实验结果显示两种方法均未达到理想的检测效能，但对过程的深入探索和对问题的细致分析，为未来工作的改进方向提供了重要的参考。

5.2 算法选择与原理

5.2.1 算法一：基于自编码器 (Autoencoder, AE) 的异常检测

本研究构建了一个基于 PyTorch 框架的多层前馈神经网络自编码器模型。

模型架构： 自编码器由编码器（Encoder）将输入数据压缩至低维潜在空间，及解码器（Decoder）尝试从该潜在表示重构原始输入构成。具体网络配置如下：输入层维度为 25（与预处理后特征数一致），编码器包含两层线性变换（25-16, 16-8），解码器同样包

含两层线性变换（8-16, 16-25），各层间使用 ReLU 激活函数，输出层无激活函数。瓶颈层维度设为 8。

"PyTorch 自编码器模型定义 (ae.py)"

```
class Autoencoder(nn.Module):
def __init__(self, input_dim , encoding_dim_1 , encoding_dim_2):
super(Autoencoder , self).__init__()
# Encoder
self.encoder = nn.Sequential(
nn.Linear(input_dim , encoding_dim_1),
nn.ReLU(True),
nn.Linear(encoding_dim_1 , encoding_dim_2),
nn.ReLU(True) # 瓶 颈 层
)
# Decoder
self.decoder = nn.Sequential(
nn.Linear(encoding_dim_2 , encoding_dim_1),
nn.ReLU(True),
nn.Linear(encoding_dim_1 , input_dim),

)
def forward(self, x):
x = self.encoder(x)
x = self.decoder(x)
return x
```

训练与检测机制： 自编码器在非欺诈交易数据 `X_train` 上进行训练，优化目标为最小化均方误差（MSE Loss），采用 Adam 优化器，学习率为 1×10^{-3}，批处理大小 256，训练 20 轮。欺诈检测时，计算测试样本的重构误差。正常样本因与训练数据模式相符，预期重构误差较低；欺诈样本则因模式偏离而产生较高重构误差。通过在测试集上绘制精确率-召回率曲线，选取使 F1 分数最大的重构误差作为区分欺诈与正常的最佳阈值。

5.2.2 算法二：基于 K-Means 聚类的异常检测

K-Means 聚类算法作为另一种无监督异常检测手段被应用于本研究。

原理与应用： K-Means 算法在非欺诈交易数据 `X_train` 上进行拟合，目的是确定代表正常交易模式的 K 个簇中心（本研究预设 K=10）。欺诈检测时，计算测试样本至所有 K 个簇中心的距离，并将样本至其最近簇中心的距离定义为其" 异常得分"。正常样本预计其异常得分较低，而欺诈样本则因远离所有" 正常" 簇中心而具有较高的异常得分。同样地，通过在测试集上最大化 F1 分数来确定最佳的距离阈值，用以区分欺诈与正常交易。

5.3 实验设置与评估指标

实验在具备 CUDA 支持的设备上优先使用 GPU（主要针对自编码器），否则在 CPU 上运行。模型性能的评估统一采用精确率、召回率、F1 分数、AUC-ROC 以及混淆矩阵。

5.4 结果与分析

5.4.1 自编码器 (AE) 实验结果

基于精确率-召回率曲线分析，选取的最佳重构误差阈值为 **0.036229**，在此阈值下达成的 F1 分数为 **0.6667**。使用此阈值对测试集进行分类，得到的详细性能报告见表 1。对应的混淆矩阵如图 1 所示。计算得到的 AUC-ROC 值为 **0.5002**。

表 5 自编码器 (AE) 分类报告 (基于 F1 最优阈值)

	precision	recall	f1-score	support
Non-Fraud	0.00	0.00	0.00	179553
Fraud	0.50	1.00	0.67	179553
accuracy			0.50	359106
macro avg	0.25	0.50	0.33	359106
weighted avg	0.25	0.50	0.33	359106

5.4.2 K-Means 聚类实验结果

K-Means 模型在测试集上的异常得分（至最近簇中心的距离）分布如图 18所示。选取的最佳距离阈值为 **1.661762**，在此阈值下达成的 F1 分数为 **0.6667**。分类性能报告见表 6，混淆矩阵如图 19所示。计算得到的 AUC-ROC 值为 **0.5312**。

表 6 K-Means 分类报告 (基于 F1 最优阈值)

	precision	recall	f1-score	support
Non-Fraud	0.00	0.00	0.00	179553
Fraud	0.50	1.00	0.67	179553
accuracy			0.50	359106
macro avg	0.25	0.50	0.33	359106
weighted avg	0.25	0.50	0.33	359106

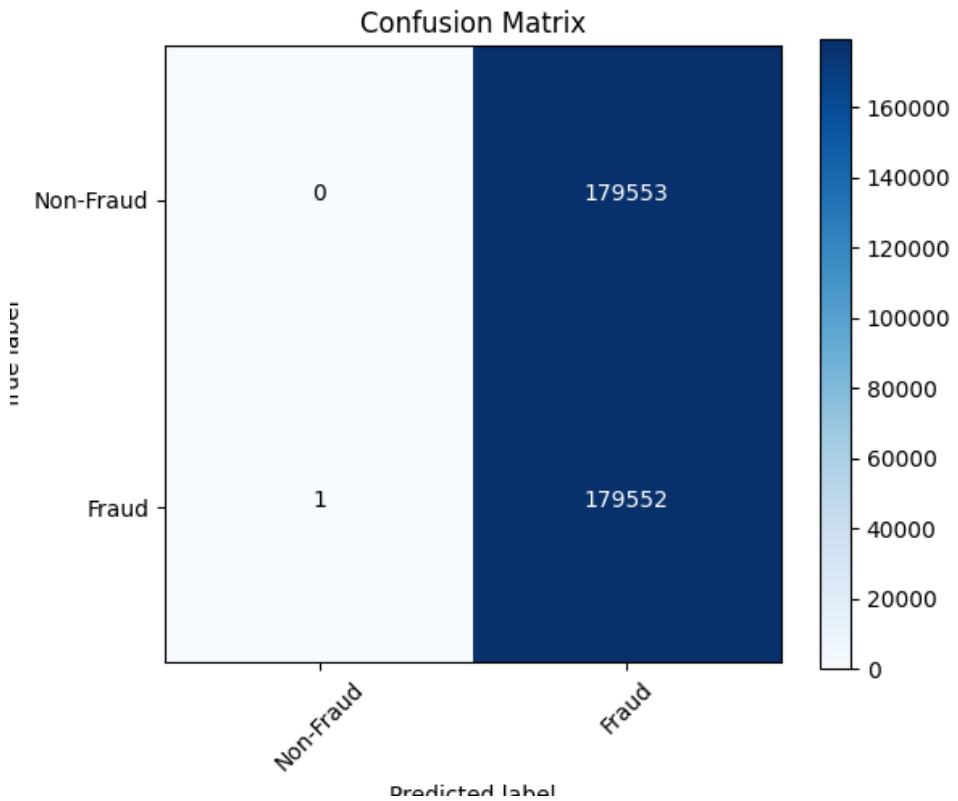


图 17 自编码器 (AE) 混淆矩阵 (基于 F1 最优阈值)

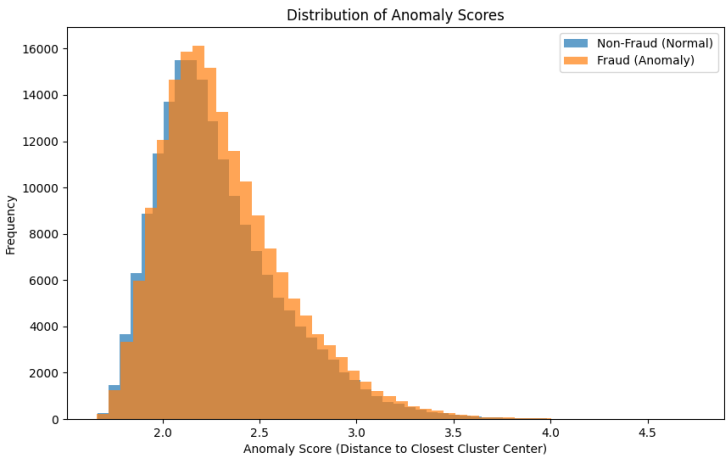


图 18 K-Means 异常得分分布图 (欺诈 vs 非欺诈)

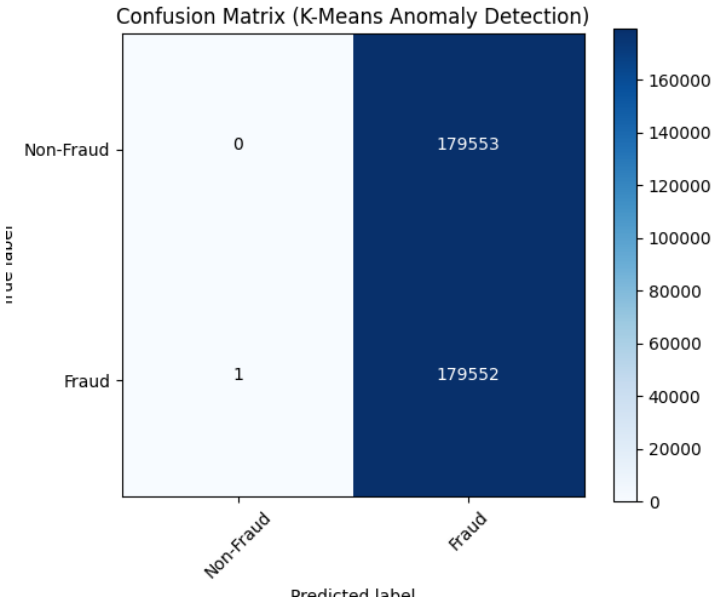


图 19 K-Means 混淆矩阵 (基于 F1 最优阈值)

5.4.3 实验结果讨论

实验结果清晰表明，当前配置下的自编码器与 K-Means 模型在所用数据集上的欺诈检测性能均显著低于预期。两种方法对于非欺诈类别的精确率、召回率及 F1 分数几乎为零，意味着模型未能有效识别出任何非欺诈样本。整体准确率均为 0.50，AUC-ROC 值亦徘徊于 0.5 附近（AE: 0.5002, K-Means: 0.5312），其性能表现与随机猜测无异。

尽管两种方法均获得了针对欺诈类别 0.67 的 F1 分数，但鉴于其对非欺诈类别的完全失效，此指标的实际意义有限。这可能指示了当前数据集特征对区分欺诈与非欺诈行为的能力不足，或者欺诈模式与正常模式在特征空间中存在高度重叠，使得基于重构误差或聚类距离的异常检测方法难以建立清晰的判别边界。此外，模型本身的局限性，如自编码器对所有类型异常的普适敏感性而非对特定欺诈的针对性，以及 K-Means 对数据簇形状和密度的假设，也可能对结果产生影响。未来的改进方向需从深化数据理解、优化特征工程、提升数据质量控制，以及探索更适应数据特性的模型或算法入手。

6 时间序列预测 (Transaction Time Series Forecasting)

本章节旨在探索针对特定交易特征（如 `device_used`, `location`, `payment_channel`）进行时间序列预测的可行性与效果。考虑到数据集的规模特性及模型构建的复杂性，本研究选取了深度学习方法与梯度提升决策树（GBDT）两类主流算法进行模型构建与比较分析。

6.1 问题定义与评价指标

时间序列预测任务的核心目标是基于历史数据中其他相关特征的变化趋势，对选定的目标特征（例如特定设备、地点或支付渠道的交易频次或欺诈发生率）在未来一段时间内的状态进行预测。模型的性能将通过一系列常用的评价指标进行衡量，例如准确率（针对分类型预测）、损失值（如均方误差 MSE，针对回归型预测）以及 F1 分数（综合评估分类性能）等，以全面评估其预测能力。

6.2 算法选择与原理

为应对时间序列预测的挑战，并兼顾数据量及模型构建的实际考量，本研究采用了深度学习与梯度提升决策树（GBDT）两类算法。

数据预处理阶段，针对预测任务的特性进行了相应调整。对于基数较低的目标预测特征（如设备类型等，通常具有有限的几种取值），采用了独热编码（One-Hot Encoding）将其转换为稀疏向量表示。而对于基数较高的其他辅助特征（如交易双方账户 `sender_account`, `receiver_account`），在深度学习模型中，考虑在编码后引入额外的嵌入层（Embedding Layer）进行降维和特征表示学习。时间戳（`timestamp`）特征被转换为以秒为单位的数值，并进行了适当的缩放处理（如除以 10^8）。`is_fraud` 特征则直接将布尔值映射为 0 和 1 的整数。交易 ID（`transaction_id`）因其对预测任务贡献有限而被舍弃。

6.2.1 算法一：基于深度学习的回归（或分类）方法

深度学习模型凭借其强大的非线性模式捕捉能力，在时间序列分析领域展现出显著优势。在本研究中，我们构建了一个多输出的深度神经网络模型，用以同时预测多个目标特征。

模型架构与训练： 模型利用 PyTorch 框架实现，其核心组件包括嵌入层（用于处理高基数类别型特征如账户 ID）和多个并行的分支网络（Branches），每个分支负责预测一个特定的目标特征（如 `device_used`, `location`, `payment_channel`）。数据通过 PyTorch 的 ‘Dataset‘ 和 ‘Dataloader‘ 进行封装和批量加载。训练集与测试集按 4:1 的比例划分。每

个分支网络通常包含若干线性层（Linear Layers）、归一化层（Batch Normalization）以及激活函数（如 ReLU），并可能引入 Dropout 层以防止过拟合。例如，一个典型的分支结构可能如下：线性层 → BatchNorm1d → ReLU → Dropout → 线性层 → BatchNorm1d → ReLU → Dropout → 输出线性层。模型的整体输入由原始的多样化特征（经过嵌入和独热编码等处理）拼接而成。针对 one-hot 编码的输出目标，采用交叉熵损失函数（Cross-Entropy Loss）进行优化，并使用 Adam 优化器（具有动态调整学习率的特性）。训练过程中的各项参数（如各分类任务的损失值）通过 TensorBoard 进行监控与记录。 因为我们要求的输出为 one-hot 编码，因此我们采用交叉熵损失函数以及 Adam

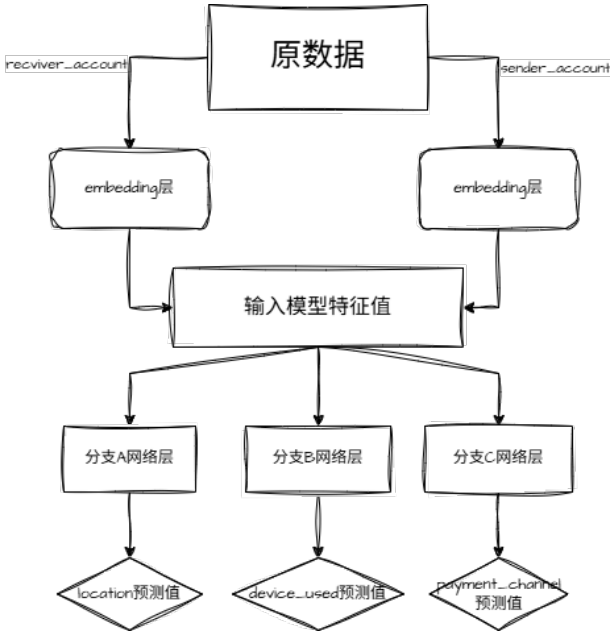


图 20 实验流程

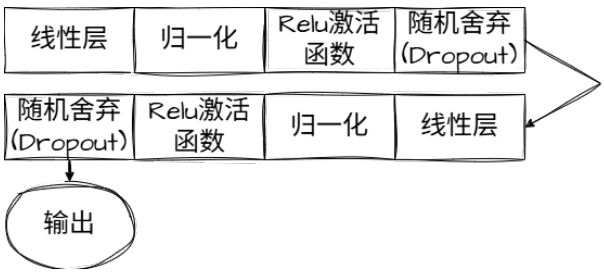
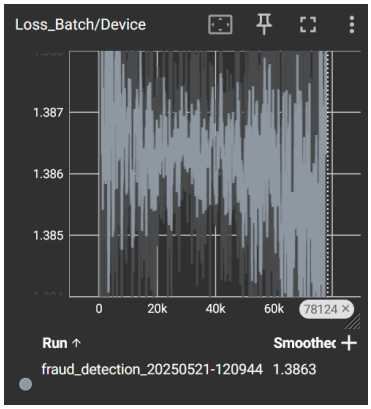


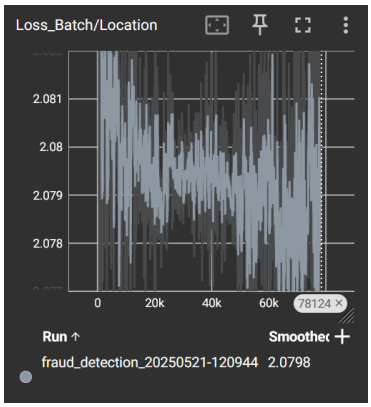
图 21 实验流程 2

优化器（动态改变学习率）。然后我们开始模型的训练，在训练过程中将各参数记录在 tensorboard 中，如下图，我们记录了各个分类任务的损失值变化：

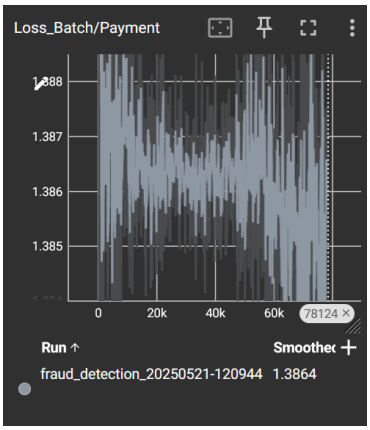
在模型训练的迭代过程中，损失函数值的变化显示出较大的不稳定性，且在训练结束时，其绝对降幅有限，收敛过程亦不甚理想。这一表现与先前 EDA 章节中关于数据质量欠佳的发现可能存在因果联系。尽管我们一度考虑应用残差网络架构来确保模型输出与输入数据的一致性，但实践证明此举收效不大。故此，我们坚持采用通过增加隐藏



a device loss



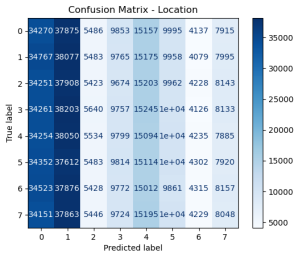
b location 数据训练损失值



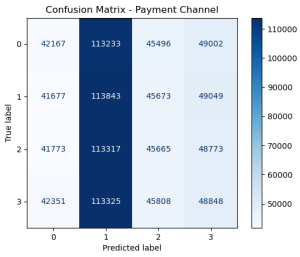
c payment channel 数据训练损失值

图 22 其他交易特征展示数据的总体样貌

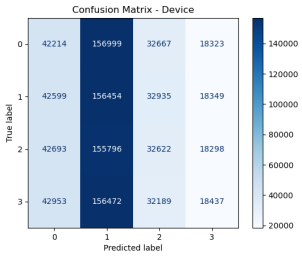
层数量和激活函数配置的策略，以期提升模型的拟合与学习复杂模式的潜力。接下来将展示各预测特征在该模型下的混淆矩阵。



a location 预测混淆矩阵



b payment channel 预测混淆矩阵



c device used 预测混淆矩阵

图 23 其他交易特征展示数据的总体样貌

可以看出模型对于某些特征值仿佛具有偏好，但考虑到我们在进行模型训练之前已经对数据进行了归一化处理，因此我们无法得知该现象产生的原因。

6.2.2 算法二：基于梯度提升决策树 (GBDT) 的预测实现

梯度提升决策树（GBDT）作为一种集成学习算法，在各类预测任务中均表现出色。本研究采用 LightGBM 库来实现 GBDT 模型。LightGBM 以其高效的训练速度和内存占用而闻名。

核心机制与应用： LightGBM 的核心运行机制包括：

- **梯度提升决策树 (GBDT)**：通过迭代地训练一系列决策树，每棵新树都致力于修正先前所有树集成起来的预测误差（对于多分类任务，通常是基于梯度的伪残差）。

- **残差学习**：新生成的树主要学习并拟合前一轮模型预测结果与真实值之间的差异。

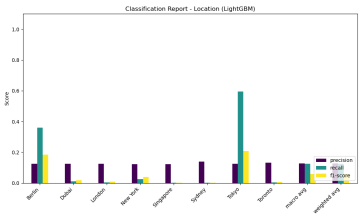
- **叶子生长策略 (Leaf-wise Growth)**：与传统的按层生长（Level-wise）不同，LightGBM 优先分裂那些能带来最大损失减少的叶子节点。这种策略通常能更快地达到较高的精度，但也需要通过参数（如 `max_depth`, `min_child_samples`）来控制过拟合。

- **并行学习支持**：LightGBM 内置了高效的并行学习机制。

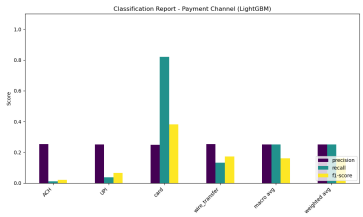
针对 `location`, `device_used`, `payment_channel` 这三个目标变量，分别训练了独立的 LightGBM 分类模型。对每个目标：

1. 复制通用的模型参数，并根据当前目标类别的数量设置 `num_class`。
2. 实例化 `lgb.LGBMClassifier`。
3. 使用训练数据进行模型拟合（`fit`），并在验证集（`eval_set`）上进行性能监控，启用早停机制（`early_stopping_rounds`）以防止过拟合（例如，若验证集上的 `multi_logloss` 在 20 轮内没有改善则停止训练）。
4. 对于类别型特征，通过 `categorical_feature` 参数告知 LightGBM，以便其能利用内置的高效处理方法。
5. 训练完成后，模型被存储。在预测阶段，对测试集进行预测得到编码后的结果，再使用相应的 `LabelEncoder` 将预测结果反向转换为原始的类别标签。

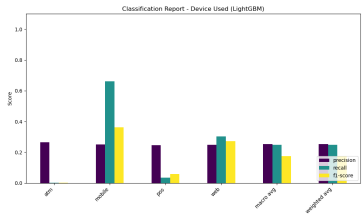
在模型成功训练之后，我们可以通过以下指标对模型进行评估：模型对于三个分类



a location 指标



b payment channel 预测混淆矩阵



c Device classification

图 24 其他交易特征展示数据的总体样貌

任务以及各个特征具有较好的区分度，尽管在某些情况下性能表现不算十分完美，但该模型也具备在特定情况下完成任务的能力。

6.3 结果与分析

在对两种模型进行评估后，我们观察到它们的准确率表现相近。然而，深度学习模型在某些方面展现出一定的优势，尽管其在训练后期损失值的收敛可能不如预期那般极致。这或许与数据集的内在复杂性、数据量以及模型当前的表达能力有关。在数据量相

对较小或对训练速度有较高要求的场景下，基于 LightGBM 的梯度提升决策树因其运行效率高、对数据要求相对较低的特性，可能成为一个更具吸引力的选择。反之，当数据量充足且模型有足够的潜力去学习复杂模式时，深度学习方法经过进一步的调优和结构优化，可能更适合。

7 总结与展望 (Conclusion and Future Work)

7.1 研究总结

本研究围绕金融交易欺诈检测这一核心议题，系统性地设计、实现并评估了一个融合多种机器学习及深度学习算法的检测框架。研究工作全面覆盖了从初始的数据探索与预处理，到针对不同欺诈检测范式（多类别欺诈分类、欺诈异常检测、以及与欺诈相关的交易特征时间序列预测）的模型构建、训练与性能分析。在多类别欺诈检测部分，对比了 XGBoost 与 CatBoost 两种主流梯度提升算法在识别与欺诈相关的设备类型上的表现。在欺诈异常检测部分，探索了基于自编码器（Autoencoder）和 K-Means 聚类的无监督方法在区分正常与欺诈交易方面的潜力。在时间序列预测部分，则针对特定交易特征（如设备、地点、支付渠道）的未来状态，分别构建并评估了基于深度学习和梯度提升决策树（GBDT）的预测模型。整个研究过程不仅关注模型最终的性能指标，也深入分析了不同算法的原理特性、参数敏感性以及在特定任务场景下的适应性。

7.2 结论

通过本研究的系列实验与分析，可以得出以下主要结论：

1. **探索性数据分析 (EDA) 的重要性：**EDA 阶段揭示了所用 AI 生成数据集在特征区分度及潜在数据质量方面存在的挑战，这直接影响了后续模型的性能表现，强调了在任何建模任务开始前进行彻底数据洞察的必要性。

2. **多类别欺诈（特征）检测的算法差异：**在基于设备类型进行多类别欺诈特征识别的任务中，XGBoost 与 CatBoost 在经过参数调优后，整体性能表现相近，但在特定设备类型的识别上各有微弱侧重。算法选择应结合具体业务对误报、漏报的容忍度以及计算资源等因素综合考量。

3. **无监督异常检测的挑战：**基于自编码器和 K-Means 的欺诈异常检测方法，在当前数据集和实验设置下，未能达到理想的区分效果，其性能接近随机猜测。这凸显了当欺诈模式与正常模式在特征空间中高度重叠或特征区分能力不足时，传统无监督异常检测方法可能面临的困境。

4. **时间序列预测模型的适用性：**在对特定交易特征进行时间序列预测时，深度学习模型和基于 GBDT（LightGBM）的模型均展现出一定的预测能力，准确率表现相

近。深度学习模型在捕捉复杂非线性关系方面具有潜力，而 GBDT 模型则在训练效率和对数据要求方面更具优势，尤其适用于数据量相对较小的场景。

5. **数据质量与特征工程的核心作用**：贯穿所有实验的核心发现是，数据质量和特征工程的优劣对模型性能起着决定性作用。若原始数据本身难以有效区分目标类别，即使采用先进的算法也难以取得突破性进展。

7.3 不足与展望

尽管本研究进行了一系列有益的探索，但仍存在一些局限性，并为未来的工作指明了方向：

1. **数据源的局限性**：本研究基于 AI 生成的交易数据，其与真实世界金融交易数据的分布特性和复杂性可能存在差异。未来研究应尽可能获取并使用经过脱敏处理的真实金融交易数据，以提升模型的现实适用性。

2. **特征工程的深化**：当前的特征工程主要集中在基础变换和部分统计特征的构建。未来可以探索更复杂的特征交互、引入外部数据源（如社交网络信息、地理位置服务数据等），以及利用图神经网络等技术挖掘交易网络中的关联特征，以期发现更具判别力的信号。

3. **模型参数调优与集成策略**：本研究中各模型的参数调优主要依赖于经验设置和部分自动化搜索。未来可以采用更系统化的超参数优化方法（如贝叶斯优化、遗传算法等）。同时，可以进一步探索不同模型（包括不同阶段、不同类型的模型）的集成策略（如 Stacking、Blending），以期获得更鲁棒和更高性能的检测系统。

4. **对异常检测方法的进一步探索**：针对无监督异常检测效果不佳的问题，未来可以尝试更先进的异常检测算法，如基于生成对抗网络（GAN）的方法、基于图的异常检测方法，或结合半监督学习的思路，利用少量已标记的欺诈样本指导模型的学习。

5. **模型可解释性的提升**：虽然部分 GBDT 模型可以提供特征重要性排序，但深度学习模型的可解释性仍是一个挑战。未来研究可以引入 LIME、SHAP 等模型解释技术，增强对模型决策过程的理解，这对于金融风控等高风险应用场景至关重要。

6. **动态适应与在线学习**：金融欺诈模式不断演变，要求欺诈检测系统具备动态适应能力。未来可以研究在线学习或增量学习机制，使模型能够持续从新的数据流中学习并更新，以应对层出不穷的新型欺诈手段。

7. **成本敏感学习的引入**：在欺诈检测中，不同类型的错误（误报和漏报）通常具有不同的业务成本。未来可以在模型训练的损失函数中引入成本敏感学习，使模型更侧重于最小化整体的风险成本。

综上所述，金融欺诈检测是一个持续演进的领域。本研究为基于多样化机器学习算法构建欺诈检测系统提供了一个实践案例和分析视角，未来的工作将致力于克服现有局限，并结合新兴技术不断提升检测系统的智能化水平和实战效能。

7.4 成员贡献

表 7 团队成员分工表

组员	职务	任务
沈文彬	组长	时间序列预测 (深度学习)
胡可秦	组员	XGBoost 的多类欺诈检测
宋浩宇	组员	欺诈异常检测的深度学习实现
朱信宇	组员	Catboost 实现多类欺诈检测
牟赞斐	组员	K-means 聚类实现
全体成员	全体成员	EDA (探索性数据分析)、时间序列预测 (LightGBM 实现)